МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛАБОРАТОРНА РОБОТА №2

з дисципліни «Тестування та контроль якості (QA) вбудованих систем»

Тема: Дослідження засобів канального рівня моделі OSI.
Протокол ARP

Варіант QA-завдання: 2

Виконав:
студент групи ІК-33
Волянський Максим

Київ — 2026

ЗМІСТ

Щоб оновити номери сторінок у змісті: правою кнопкою по змісту → «Оновити поле» (Update Field).

1. Мета роботи та вихідні дані

1.1. Посилання на репозиторій

Репозиторій з результатами виконання: https://github.com/<ваш-логін>/qa-lab2-arp

У репозиторії розміщено: скрипт налаштування мережного оточення (`setup_network.sh`), файли захопленого трафіку Wireshark (`.pcapng`) для кожного експерименту, скриншоти та тестову документацію (`TEST_CASES.md`).

1.2. Мета роботи

Вивчити технологію отримання MAC-адреси на канальному рівні мережі за відомою IP-адресою; розібратися із засобами підтримки протоколу ARP на рівні операційної системи Linux і навчитися переглядати ARP-кеш, видаляти з нього записи та очищувати його; навчитися аналізувати мережний трафік за допомогою програми Wireshark; розробити тестову документацію (Test Cases) для перевірки роботи протоколу ARP.

1.3. Визначення варіанту

Номер у списку групи — 5, тобто дві останні цифри: 05. Остача від ділення 05 на 3: 5 mod 3 = **2**. Отже, основне QA-завдання — **варіант 2**: додати статичний запис в ARP-таблицю, після цього пропінгувати інший пристрій і відстежити трафік у Wireshark. Додатково (на додаткові бали) описано тест-кейси для варіантів 0 і 1 та три власні тест-кейси.

1.4. Обладнання та середовище виконання

Роботу виконано на ОС Ubuntu 24.04 LTS (ядро Linux 6.18, x86_64). Методичні вказівки допускають заміну двох фізичних ноутбуків двома віртуальними машинами у спільній внутрішній мережі. У цій роботі використано рівноцінний, але легший варіант: два **мережні простори імен** Linux (*network namespaces*) `pc1` та `pc2`. Кожен простір має повністю власний мережний стек: інтерфейси, IP-адреси, таблицю маршрутизації та **власний ARP-кеш**, тобто з погляду мережі це два окремі комп'ютери. Їх з'єднано **парою віртуальних Ethernet-інтерфейсів** (*veth pair*): кадр, надісланий в один кінець, виходить з іншого, як через кабель (схема на рис. 1). Перевага такого стенду в тому, що в ньому немає стороннього трафіку браузера, оновлень тощо, тому в Wireshark видно лише досліджувані кадри.

Компонент	Версія / значення
ОС	Ubuntu 24.04 LTS, ядро 6.18 (x86_64), bash
Wireshark / tshark	4.2.2
iproute2 (`ip link`, `ip neigh`)	6.1.0
net-tools (`arp`)	2.10
iputils (`ping`)	20240117
arping	2.24
PC1 (Host)	netns pc1, інтерфейс veth-pc1, IP 192.168.10.1/24, MAC 7e:83:38:fc:b8:cf
PC2 (Client)	netns pc2, інтерфейс veth-pc2, IP 192.168.10.2/24, MAC 2e:c0:75:30:29:a5

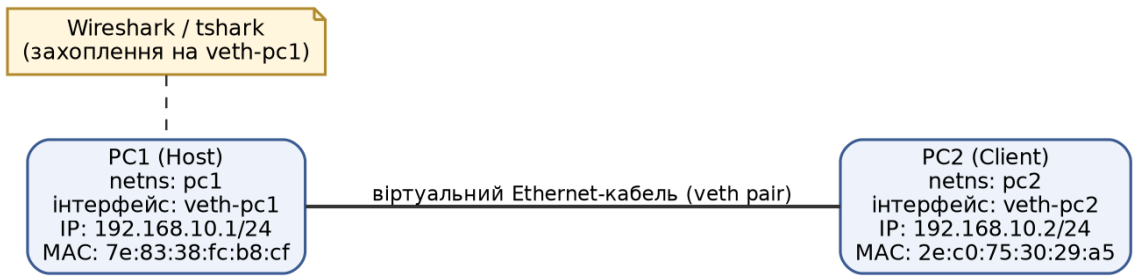


Рис. 1. *Схема тестового мережного оточення*

2. Теоретичні відомості

2.1. Канальний рівень і MAC-адреса

Канальний рівень (Data Link, рівень 2 моделі OSI) відповідає за передавання кадрів між вузлами **однієї** фізичної мережі (сегмента). Вузли на цьому рівні адресуються не IP-адресами, а **MAC-адресами**: 48-бітними (6 байтів) апаратними адресами мережних інтерфейсів, які записують шістнадцятковими байтами через двокрапку, наприклад `7e:83:38:fc:b8:cf`. Перші три байти глобально унікальної MAC-адреси — OUI (ідентифікатор виробника), останні три — номер інтерфейсу.

Два молодші біти першого байта мають спеціальне значення: **I/G-біт** (0 — індивідуальна адреса, 1 — групова), **U/L-біт** (0 — глобально унікальна, 1 — локально призначена). Адреса `ff:ff:ff:ff:ff:ff` — широкомовна (broadcast): такий кадр приймають усі вузли сегмента. MAC-адреси віртуальних інтерфейсів у цій роботі (`7e:…`, `2e:…`, `02:…`) локально призначені, що Wireshark і показує в полі «LG bit: Locally administered address».

Формат кадру Ethernet II: MAC одержувача (6 байтів), MAC відправника (6 байтів), поле EtherType (2 байти), дані (46–1500 байтів) і контрольна сума FCS (4 байти). Поле EtherType визначає вкладений протокол: `0x0800` — IPv4, **0x0806 — ARP**.

2.2. Протокол ARP

ARP (*Address Resolution Protocol*, RFC 826) — протокол, що за відомою IPv4-адресою вузла в локальній мережі визначає його MAC-адресу. Він потрібен тому, що IP-пакет (рівень 3) для фактичного передавання загортається в Ethernet-кадр (рівень 2), а для кадру потрібна MAC-адреса одержувача. ARP працює на межі рівнів 2 і 3 і не виходить за межі широкомовного домену: для вузла з іншої підмережі визначається MAC-адреса шлюзу.

Обмін складається з двох повідомлень:

- **ARP Request** (Opcode = 1) — *широкомовний* кадр (Dst MAC = `ff:ff:ff:ff:ff:ff`) із питанням «Who has 192.168.10.2? Tell 192.168.10.1». Поле Target MAC заповнене нулями, бо саме його й шукаємо.
- **ARP Reply** (Opcode = 2) — *одноадресний* (unicast) кадр від власника IP-адреси: «192.168.10.2 is at 2e:c0:75:30:29:a5».

Таблиця 2.1. Формат ARP-пакета (Ethernet + IPv4) і значення полів із захопленого кадру ARP Request

Поле	Розмір	Призначення	Значення в кадрі №1
Hardware type (HTYPE)	2 Б	Тип канального рівня	1 (Ethernet)
Protocol type (PTYPE)	2 Б	Тип адреси мережного рівня	0x0800 (IPv4)
Hardware size (HLEN)	1 Б	Довжина MAC-адреси	6
Protocol size (PLEN)	1 Б	Довжина IP-адреси	4
Opcode (OPER)	2 Б	1 — request, 2 — reply	1 (request)
Sender MAC address	6 Б	MAC відправника	7e:83:38:fc:b8:cf
Sender IP address	4 Б	IP відправника	192.168.10.1
Target MAC address	6 Б	MAC цілі (у запиті невідомий)	00:00:00:00:00:00
Target IP address	4 Б	IP, для якого шукаємо MAC	192.168.10.2

Разом ARP-пакет займає 28 байтів, а з Ethernet-заголовком (14 байтів) — **42 байти**, що збігається з довжиною кадрів ARP у захопленні (стовпець Length). На фізичній лінії мережна карта доповнює такий кадр до мінімальних 60 байтів (+4 байти FCS). Захоплення велось на стороні відправника, тому доповнення не видно.

2.3. Алгоритм розпізнавання адреси

Перед відправленням кожного IP-пакета ОС перевіряє ARP-кеш. Якщо запис є, кадр формується одразу. Якщо запису немає, ОС ставить пакет у чергу, надсилає широкомовний ARP Request і чекає на Reply; за відсутності відповіді запит повторюється (у Linux за замовчуванням 3 спроби з інтервалом 1 с, параметр `mcast_solicit`), після чого запис позначається як FAILED, а застосунок отримує помилку «Destination Host Unreachable». Алгоритм показано на рис. 2.

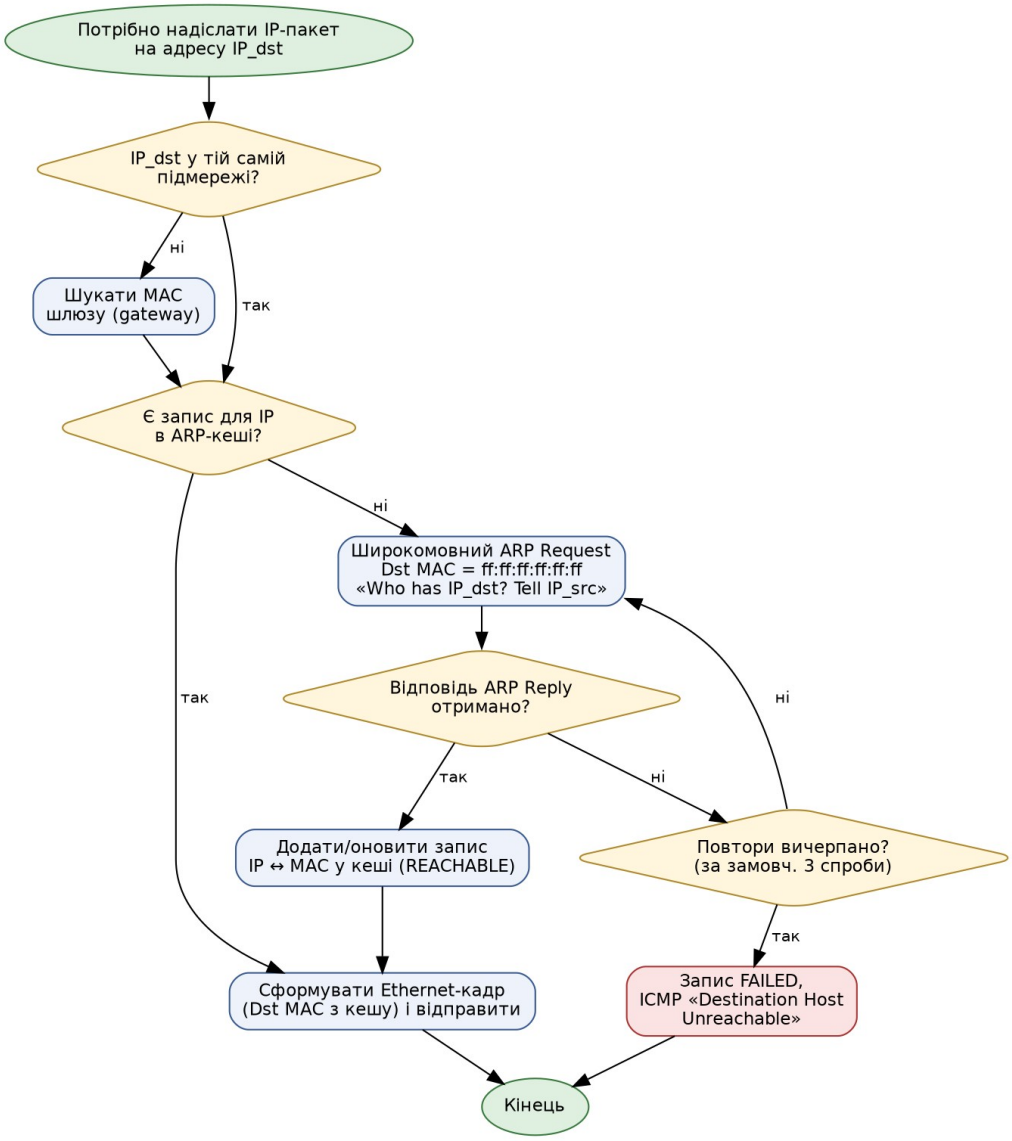


Рис. 2. *Блок-схема алгоритму визначення MAC-адреси протоколом ARP*

Важлива оптимізація: вузол, що *отримав* ARP Request, одразу запам'ятовує пару «Sender IP → Sender MAC» відправника. Тому після першого обміну обидва вузли знають MAC-адреси один одного, і PC2 не надсилає зустрічного широкомовного запиту.

2.4. ARP-кеш у Linux

Результати розпізнавання зберігаються в ARP-кеші (таблиці сусідів, *neighbour table*). Запис має стан (NUD — Neighbour Unreachability Detection), який змінюється з часом:

Таблиця 2.2. Стани записів ARP-кешу в Linux

Стан	Значення
INCOMPLETE	ARP Request надіслано, відповіді ще немає
REACHABLE	MAC підтверджено нещодавно (за замовч. ~30 с), запис використовується без перевірок
STALE	Запис застарів, але може використовуватися; при першому використанні переходить у DELAY
DELAY	Очікування підтвердження від верхніх рівнів (5 с); якщо його немає — PROBE
PROBE	Надсилаються *одноадресні* ARP Request на відомий MAC для перевірки
FAILED	Вузол не відповів на жоден запит
PERMANENT	Статичний запис, доданий вручну: не старіє і не перевіряється

Таблиця 2.3. Команди для роботи з ARP-кешем

Дія	net-tools (класична)	iproute2 (сучасна)
Переглянути кеш	`arp -a`, `arp -n`	`ip neigh show`
Видалити запис	`sudo arp -d <IP>`	`sudo ip neigh del <IP> dev <iface>`
Очистити кеш	—	`sudo ip neigh flush all`
Додати статичний запис	`sudo arp -s <IP> <MAC>`	`sudo ip neigh add <IP> lladdr <MAC> dev <iface> nud permanent`
Надіслати ARP вручну	`arping <IP>`	`arping -U -P …` — gratuitous ARP Reply

2.5. Gratuitous ARP і безпека

Gratuitous ARP — ARP-повідомлення, яке вузол надсилає *без запиту* про власну IP-адресу, щоб повідомити всім свою (нову) MAC-адресу або перевірити конфлікт IP. Отримавши такий ARP Reply, інші вузли оновлюють наявні записи. ARP не має автентифікації, тому ця ж властивість лежить в основі атаки **ARP spoofing** (підміна MAC у кешах жертв). Захистом є статичні записи для критичних вузлів, Dynamic ARP Inspection на комутаторах і моніторинг. Wireshark попереджає про таку ситуацію повідомленням «duplicate use of … detected».

2.6. Фільтри Wireshark, використані в роботі

Фільтр	Що показує
`arp`	Усі ARP-кадри
`arp || icmp`	ARP і ICMP (ping) — повна картина обміну
`arp.opcode == 1` / `arp.opcode == 2`	Лише запити / лише відповіді
`arp.src.proto_ipv4 == 192.168.10.1`	ARP-кадри, надіслані від IP 192.168.10.1
`eth.dst == ff:ff:ff:ff:ff:ff`	Широкомовні кадри
`arp.duplicate-address-detected`	Кадри з виявленим конфліктом IP/MAC

3. Хід виконання роботи

3.1. Налаштування мережного оточення

Оточення створюється скриптом `setup_network.sh` (є в репозиторії). Основні команди:

```
sudo ip netns add pc1 && sudo ip netns add pc2          # два "комп'ютери"
sudo ip link add veth-pc1 type veth peer name veth-pc2  # віртуальний кабель
sudo ip link set veth-pc1 netns pc1; sudo ip link set veth-pc2 netns pc2
sudo ip -n pc1 addr add 192.168.10.1/24 dev veth-pc1
sudo ip -n pc2 addr add 192.168.10.2/24 dev veth-pc2
sudo ip -n pc1 link set veth-pc1 up; sudo ip -n pc2 link set veth-pc2 up
sudo ip netns exec pc1 bash                             # термінал PC1
sudo ip netns exec pc1 wireshark -i veth-pc1 -k         # Wireshark на PC1
```

Перевіряємо інтерфейси обох ПК і переконуємося, що ARP-кеші порожні: `arp -a` та `ip neigh show` нічого не виводять (рис. 3, рис. 4). У першому стовпці видно MAC-адреси інтерфейсів, які далі порівнюватимемо із записами ARP-кешу.

```
maks@pc1:~$ ip -br link show veth-pc1
veth-pc1@if5     UP             7e:83:38:fc:b8:cf <BROADCAST,MULTICAST,UP,LOWER_UP>
maks@pc1:~$ ip -br addr show veth-pc1
veth-pc1@if5     UP             192.168.10.1/24
maks@pc1:~$ arp -a
maks@pc1:~$ ip neigh show
maks@pc1:~$
```

Рис. 3. *PC1: інтерфейс veth-pc1, його MAC та IP; ARP-кеш порожній*

```
maks@pc2:~$ ip -br link show veth-pc2
veth-pc2@if6     UP             2e:c0:75:30:29:a5 <BROADCAST,MULTICAST,UP,LOWER_UP>
maks@pc2:~$ ip -br addr show veth-pc2
veth-pc2@if6     UP             192.168.10.2/24
maks@pc2:~$ arp -a
maks@pc2:~$ ip neigh show
maks@pc2:~$
```

Рис. 4. *PC2: інтерфейс veth-pc2, його MAC та IP; ARP-кеш порожній*

3.2. Отримання MAC-адреси за відомою IP-адресою

На PC1 запущено захоплення трафіку на інтерфейсі `veth-pc1`. Після цього PC1 пінгує PC2 (рис. 5). До ping ARP-кеш порожній, після ping у ньому з'являється запис `192.168.10.2 at 2e:c0:75:30:29:a5` у стані REACHABLE. Отже, ОС автоматично визначила MAC-адресу PC2 за його IP.

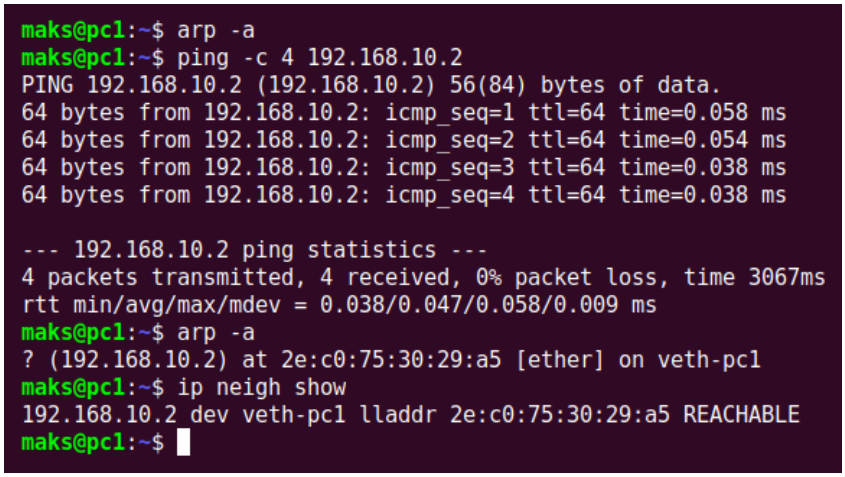


Рис. 5. *PC1: ping PC2 та поява запису в ARP-кеші*

Захоплений трафік (рис. 6) показує послідовність подій:

- **Кадр 1** — PC1 (7e:83:38:fc:b8:cf) надсилає **широкомовний** (Destination: Broadcast) ARP Request «Who has 192.168.10.2? Tell 192.168.10.1».
- **Кадр 2** — PC2 відповідає **одноадресним** ARP Reply «192.168.10.2 is at 2e:c0:75:30:29:a5» безпосередньо на MAC PC1. Між запитом і відповіддю минуло лише ~20 мкс.
- **Кадри 3–10** — чотири пари ICMP Echo request/reply. Перший ICMP-запит (кадр 3) з'являється лише після отримання ARP Reply.
- **Кадри 11–12** — через ~5 с PC2 надсилає вже **одноадресний** ARP Request на MAC PC1. PC2 дізнався MAC PC1 із запиту в кадрі 1 без підтвердження (стан STALE → DELAY), тому після 5 с очікування перевіряє запис (стан PROBE). Саме таку пару рядків наведено в методичних вказівках (рядки 13–14).

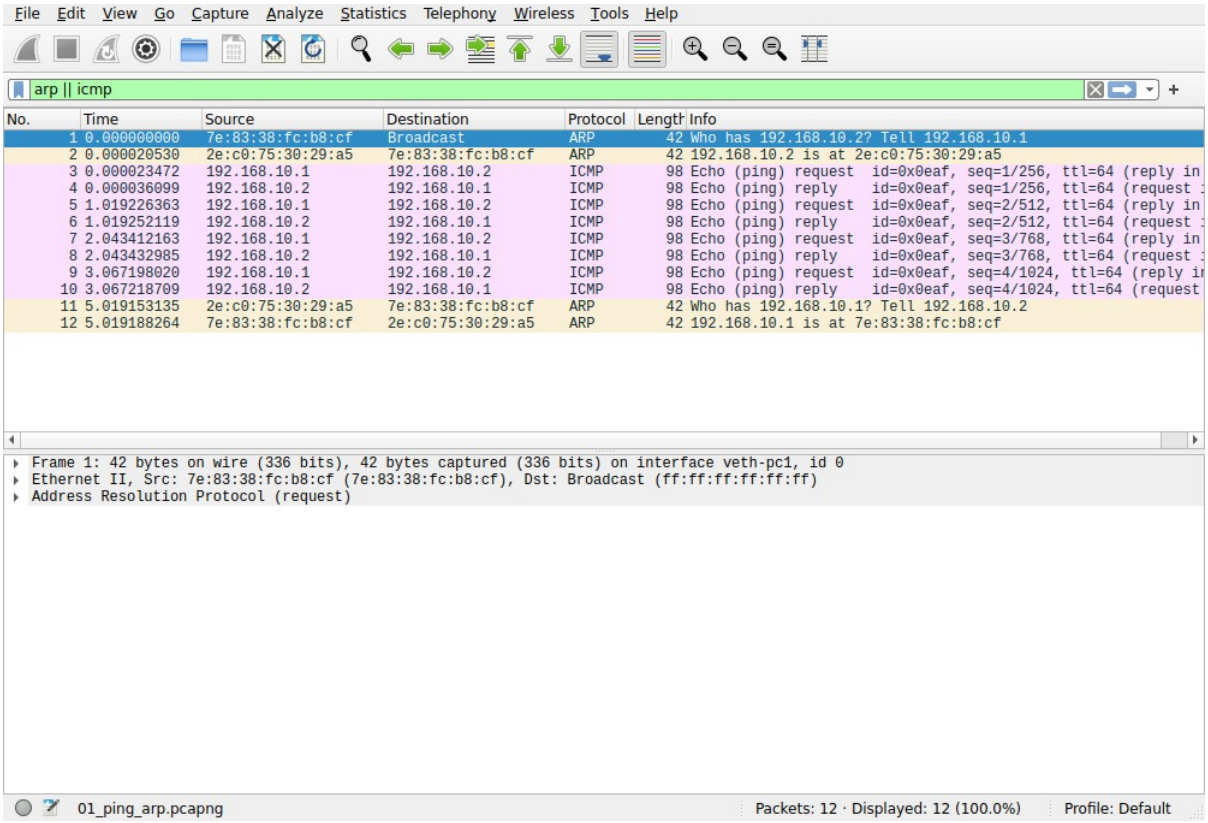


Рис. 6. *Wireshark: ARP-обмін і ICMP-трафік між PC1 та PC2 (фільтр arp || icmp)*

Детальний розбір кадру ARP Request (рис. 7): Opcode = request (1), Sender MAC/IP — адреси PC1, Target MAC = `00:00:00:00:00:00` (невідомий), Target IP = 192.168.10.2. У заголовку Ethernet Dst = `ff:ff:ff:ff:ff:ff`. У шістнадцятковому дампі видно EtherType `08 06` (ARP), а далі поля ARP-пакета в порядку з табл. 2.1.

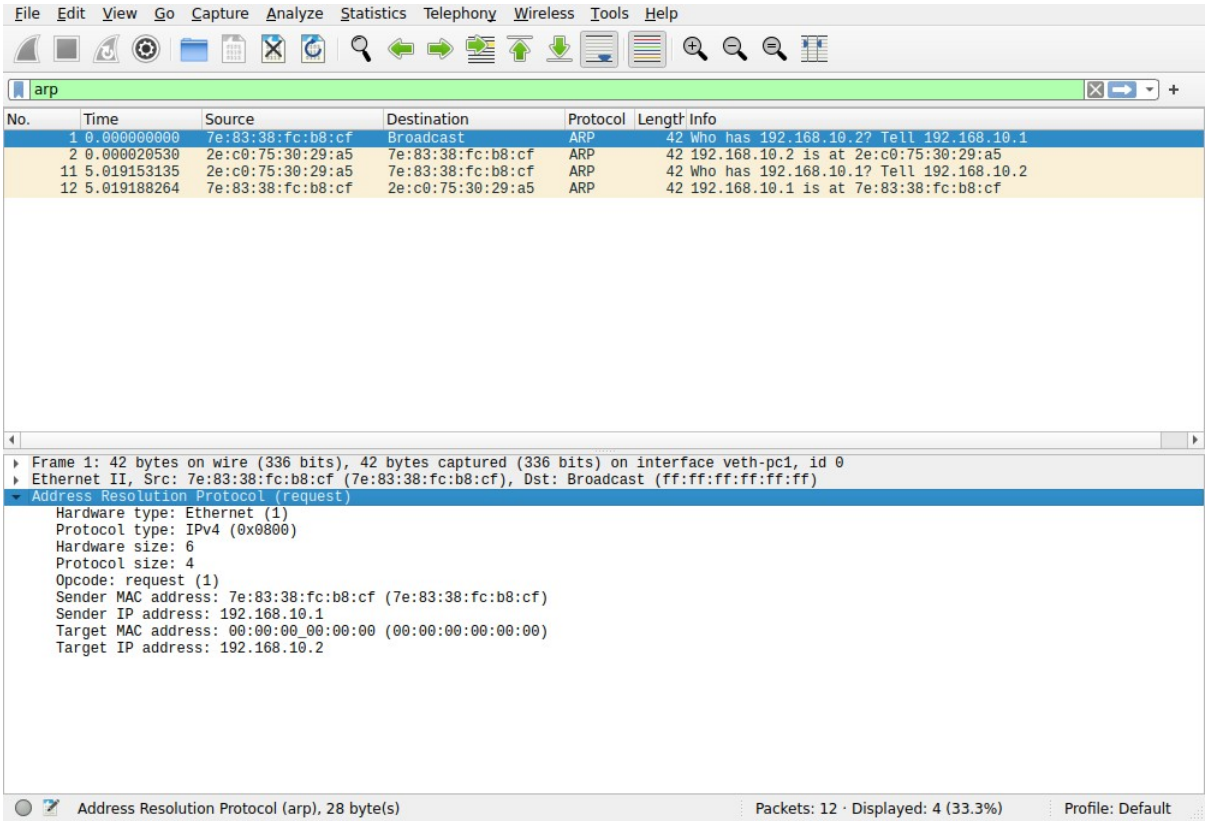


Рис. 7. *Wireshark: поля кадру ARP Request*

У кадрі ARP Reply (рис. 8) Opcode = reply (2), Sender MAC = `2e:c0:75:30:29:a5` — шукана MAC-адреса PC2, а Target — адреси PC1. Значення Sender MAC збігається із записом в ARP-кеші PC1 (рис. 5), отже кеш заповнено саме даними з ARP Reply.

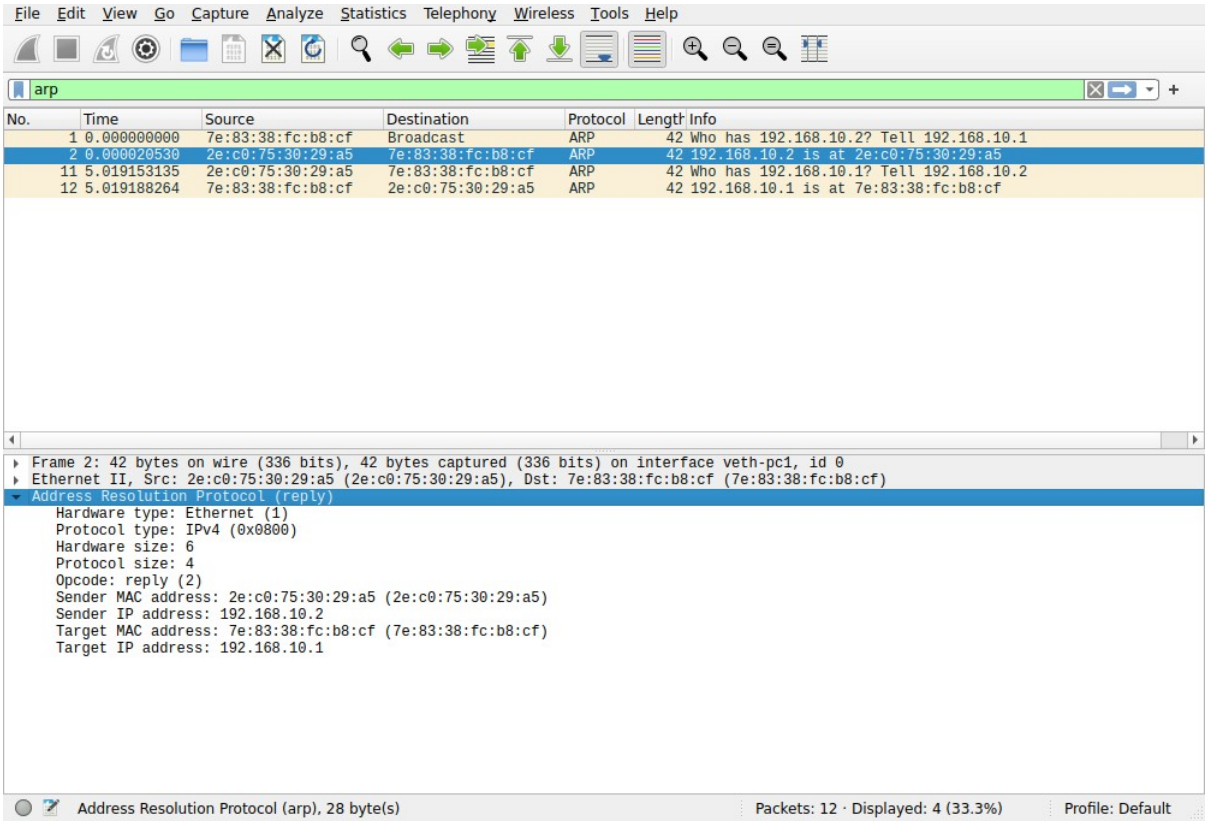


Рис. 8. *Wireshark: поля кадру ARP Reply*

ARP-кеш PC2 теж містить запис про PC1 (рис. 9), хоча PC2 сам широкомовних запитів не надсилав: запис створено з полів Sender отриманого ARP Request.

```
maks@pc2:~$ arp -a
? (192.168.10.1) at 7e:83:38:fc:b8:cf [ether] on veth-pc2
maks@pc2:~$ ip neigh show
192.168.10.1 dev veth-pc2 lladdr 7e:83:38:fc:b8:cf REACHABLE
maks@pc2:~$
```

Рис. 9. *PC2: ARP-кеш містить MAC-адресу PC1*

3.3. Перегляд, видалення та очищення ARP-кешу

На рис. 10 показано видалення запису класичною командою `arp -d` і командою `ip neigh del`, а також повне очищення кешу `ip neigh flush all`. Після кожного видалення наступний ping знову створює запис, бо ОС повторно виконує ARP-розпізнавання.

```
maks@pc1:~$ arp -a
? (192.168.10.2) at 2e:c0:75:30:29:a5 [ether] on veth-pc1
maks@pc1:~$ sudo arp -d 192.168.10.2
maks@pc1:~$ arp -a
maks@pc1:~$ ping -c 1 192.168.10.2 > /dev/null && echo "[ping виконано — запис знову з'явився]"
[ping виконано — запис знову з'явився]
maks@pc1:~$ ip neigh show
192.168.10.2 dev veth-pc1 lladdr 2e:c0:75:30:29:a5 REACHABLE
maks@pc1:~$ sudo ip neigh del 192.168.10.2 dev veth-pc1
maks@pc1:~$ ip neigh show
maks@pc1:~$ ping -c 1 192.168.10.2 > /dev/null && echo "[ping виконано]"
[ping виконано]
maks@pc1:~$ ip neigh show
192.168.10.2 dev veth-pc1 lladdr 2e:c0:75:30:29:a5 REACHABLE
maks@pc1:~$ sudo ip neigh flush all
maks@pc1:~$ ip neigh show
maks@pc1:~$
```

Рис. 10. *PC1: видалення записів (arp -d, ip neigh del) та очищення ARP-кешу (ip neigh flush all)*

3.4. Статичний запис в ARP-таблиці та ping (варіант 2)

Щоб експеримент був чистим, спочатку очищено кеш PC2 (рис. 11). Кеш PC1 після попереднього кроку вже порожній. На PC1 знову запущено захоплення трафіку.

```
maks@pc2:~$ sudo ip neigh flush all
maks@pc2:~$ ip neigh show
maks@pc2:~$
```

Рис. 11. *PC2: очищення ARP-кешу перед експериментом*

На PC1 додано статичний запис `sudo arp -s 192.168.10.2 2e:c0:75:30:29:a5`. Команда `arp -a` показує його з прапорцем **PERM**, а `ip neigh show` — зі станом **PERMANENT**. Ping проходить без втрат, а після ping запис лишається PERMANENT (рис. 12).

```
maks@pc1:~$ ip neigh show
maks@pc1:~$ sudo arp -s 192.168.10.2 2e:c0:75:30:29:a5
maks@pc1:~$ arp -a
? (192.168.10.2) at 2e:c0:75:30:29:a5 [ether] PERM on veth-pc1
maks@pc1:~$ ip neigh show
192.168.10.2 dev veth-pc1 lladdr 2e:c0:75:30:29:a5 PERMANENT
maks@pc1:~$ ping -c 4 192.168.10.2
PING 192.168.10.2 (192.168.10.2) 56(84) bytes of data.
64 bytes from 192.168.10.2: icmp_seq=1 ttl=64 time=0.035 ms
64 bytes from 192.168.10.2: icmp_seq=2 ttl=64 time=0.035 ms
64 bytes from 192.168.10.2: icmp_seq=3 ttl=64 time=0.034 ms
64 bytes from 192.168.10.2: icmp_seq=4 ttl=64 time=0.047 ms

--- 192.168.10.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3051ms
rtt min/avg/max/mdev = 0.034/0.037/0.047/0.005 ms
maks@pc1:~$ ip neigh show
192.168.10.2 dev veth-pc1 lladdr 2e:c0:75:30:29:a5 PERMANENT
maks@pc1:~$
```

Рис. 12. *PC1: додавання статичного запису і ping PC2*

Аналіз трафіку (рис. 13):

- **Кадр 1** — PC1 одразу надсилає ICMP Echo request, **без ARP-запиту**. У заголовку Ethernet Dst = `2e:c0:75:30:29:a5`, тобто MAC-адреса взята зі статичного запису.
- **Кадри 2–3** — ARP Request/Reply ініціює **PC2**: його кеш порожній, і щоб надіслати відповідь на ping, йому потрібна MAC-адреса PC1. Статичний запис діє лише на тому вузлі, де його додано.
- **Кадри 4–10** — решта ICMP-обміну без ARP.

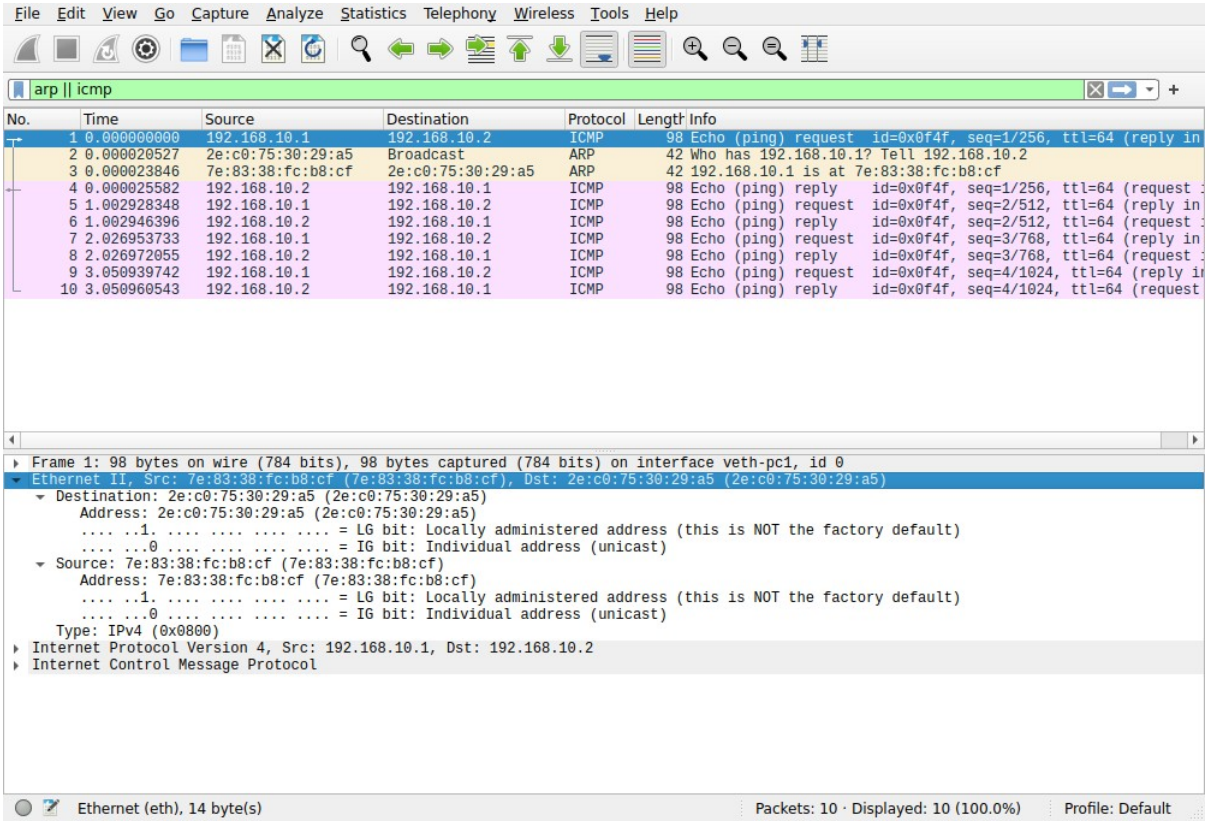


Рис. 13. *Wireshark: трафік при статичному записі на PC1, заголовок Ethernet ICMP-запиту*

Для формального підтвердження застосовано фільтр `arp.opcode == 1 && arp.src.proto_ipv4 == 192.168.10.1` (усі ARP-запити від PC1). Результат — **Displayed: 0** (рис. 14): PC1 не надіслав жодного ARP-запиту.

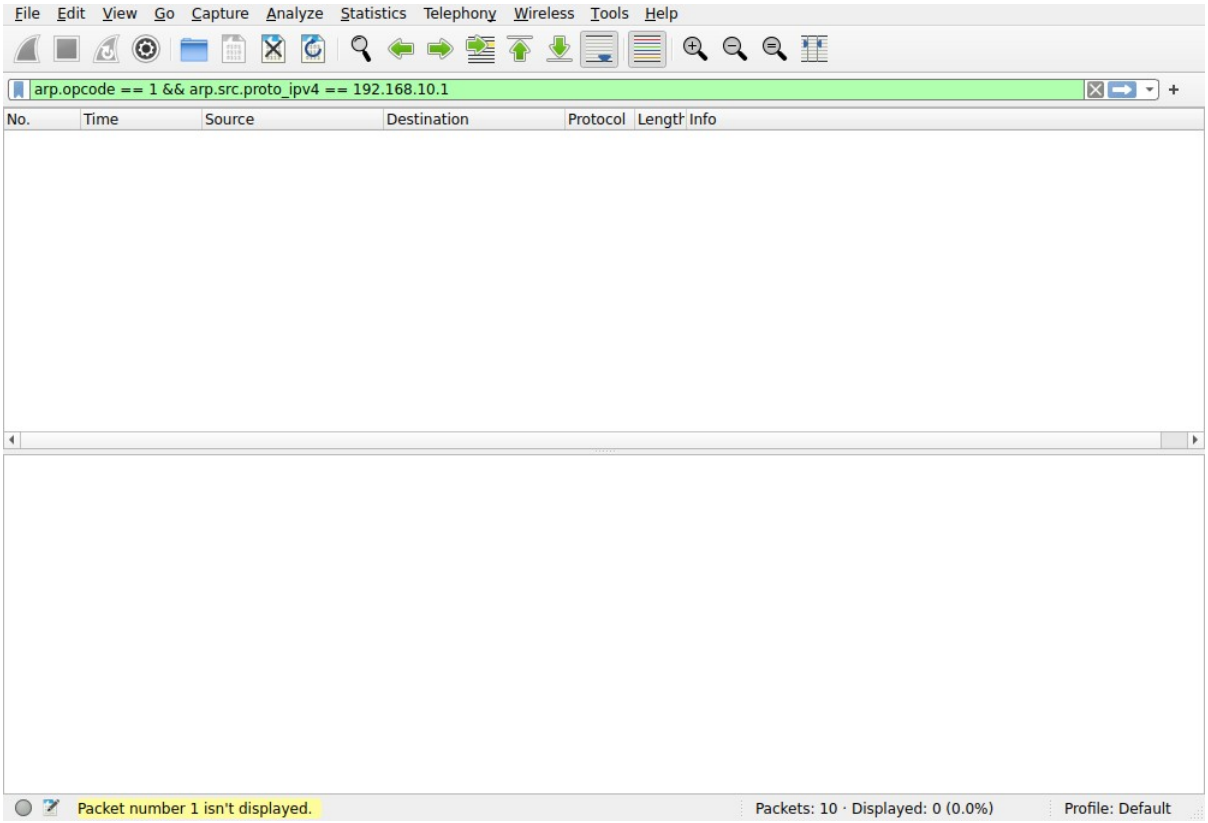


Рис. 14. *Wireshark: ARP-запитів від PC1 немає (Displayed: 0)*

Додатково перевірено, що `ip neigh flush all` **не видаляє** статичні записи: запис лишається PERMANENT, а видалити його можна лише явно, командою `arp -d` (рис. 15).

```
maks@pc1:~$ sudo ip neigh flush all
maks@pc1:~$ ip neigh show
192.168.10.2 dev veth-pc1 lladdr 2e:c0:75:30:29:a5 PERMANENT
maks@pc1:~$ sudo arp -d 192.168.10.2
maks@pc1:~$ ip neigh show
maks@pc1:~$
```

Рис. 15. *PC1: статичний запис переживає flush і видаляється лише через arp -d*

3.5. Оновлення ARP-таблиці при отриманні ARP Reply

Спочатку PC1 дізнається MAC PC2 звичайним способом: запис `2e:c0:75:30:29:a5 REACHABLE` (рис. 16). Далі на PC2 змінено MAC інтерфейсу на `02:aa:bb:cc:dd:02` і за допомогою `arping -U -P` надіслано незапитаний (gratuitous) ARP Reply з новою адресою (рис. 17). Повідомлення arping «Timeout / 100% unanswered» — очікуване: на незапитану відповідь ніхто не відповідає.

```
maks@pc1:~$ ip neigh show
maks@pc1:~$ ping -c 1 192.168.10.2
PING 192.168.10.2 (192.168.10.2) 56(84) bytes of data.
64 bytes from 192.168.10.2: icmp_seq=1 ttl=64 time=0.041 ms

--- 192.168.10.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.041/0.041/0.041/0.000 ms
maks@pc1:~$ ip neigh show
192.168.10.2 dev veth-pc1 lladdr 2e:c0:75:30:29:a5 REACHABLE
maks@pc1:~$
```

Рис. 16. *PC1: початковий запис про PC2 в ARP-кеші*

```
maks@pc2:~$ ip -br link show veth-pc2
veth-pc2@if6     UP             2e:c0:75:30:29:a5 <BROADCAST,MULTICAST,UP,LOWER_UP>
maks@pc2:~$ sudo ip link set veth-pc2 address 02:aa:bb:cc:dd:02
maks@pc2:~$ ip -br link show veth-pc2
veth-pc2@if6     UP             02:aa:bb:cc:dd:02 <BROADCAST,MULTICAST,UP,LOWER_UP>
maks@pc2:~$ sudo arping -U -P -c 1 -I veth-pc2 -S 192.168.10.2 192.168.10.2
ARPING 192.168.10.2
Timeout

--- 192.168.10.2 statistics ---
1 packets transmitted, 0 packets received, 100% unanswered (0 extra)

maks@pc2:~$
```

Рис. 17. *PC2: зміна MAC-адреси та розсилання ARP Reply*

Після цього в кеші PC1 запис **оновився** на `02:aa:bb:cc:dd:02` (стан STALE), і ping працює з новою адресою (рис. 18). У Wireshark (рис. 19) кадр 5 — ARP Reply (Opcode 2, [Is gratuitous: True]) від нового MAC. Wireshark виділив його жовтим і попередив: «Duplicate IP address detected for 192.168.10.2 … also in use by 2e:c0:75:30:29:a5», бо в одному захопленні бачив дві MAC для однієї IP. Так на практиці виявляють ARP spoofing.

```
maks@pc1:~$ ip neigh show
192.168.10.2 dev veth-pc1 lladdr 02:aa:bb:cc:dd:02 STALE
maks@pc1:~$ ping -c 2 192.168.10.2
PING 192.168.10.2 (192.168.10.2) 56(84) bytes of data.
64 bytes from 192.168.10.2: icmp_seq=1 ttl=64 time=0.070 ms
64 bytes from 192.168.10.2: icmp_seq=2 ttl=64 time=0.042 ms

--- 192.168.10.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1029ms
rtt min/avg/max/mdev = 0.042/0.056/0.070/0.014 ms
maks@pc1:~$ ip neigh show
192.168.10.2 dev veth-pc1 lladdr 02:aa:bb:cc:dd:02 DELAY
maks@pc1:~$
```

Рис. 18. *PC1: запис оновлено на нову MAC-адресу, ping успішний*

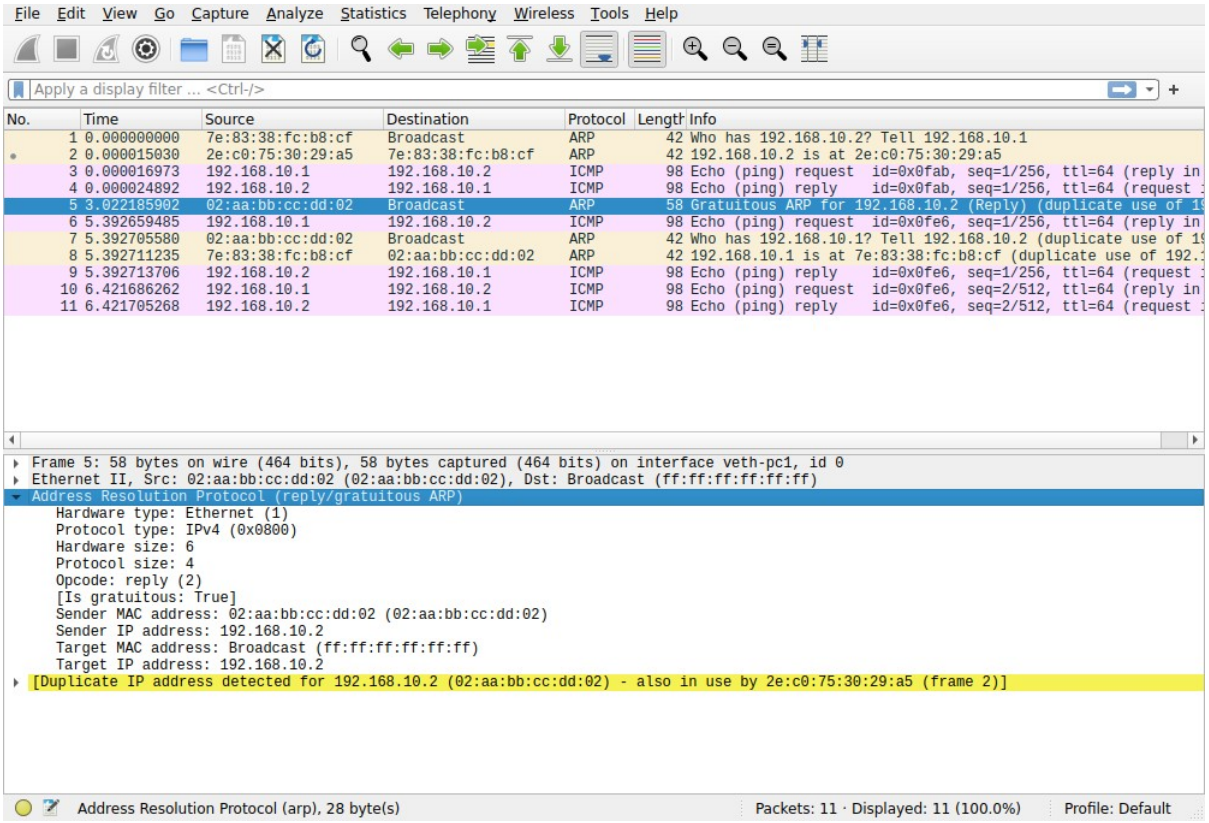


Рис. 19. *Wireshark: gratuitous ARP Reply та попередження про дублювання IP*

3.6. Додаткові (негативні) сценарії

Неіснуючий вузол. Ping на 192.168.10.99 (рис. 20) повертає «Destination Host Unreachable» від самого PC1. У Wireshark (рис. 21) видно лише три широкомовні ARP Request з інтервалом ~1 с без жодної відповіді; ICMP-кадри в мережу взагалі не виходять, бо без MAC-адреси неможливо сформувати Ethernet-кадр. У кеші лишається запис у стані FAILED.

```
maks@pc1:~$ ping -c 3 192.168.10.99
PING 192.168.10.99 (192.168.10.99) 56(84) bytes of data.
From 192.168.10.1 icmp_seq=1 Destination Host Unreachable
From 192.168.10.1 icmp_seq=2 Destination Host Unreachable
From 192.168.10.1 icmp_seq=3 Destination Host Unreachable

--- 192.168.10.99 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2041ms
pipe 3
maks@pc1:~$ ip neigh show
192.168.10.99 dev veth-pc1 FAILED
maks@pc1:~$
```

Рис. 20. *PC1: ping неіснуючої адреси, запис FAILED*

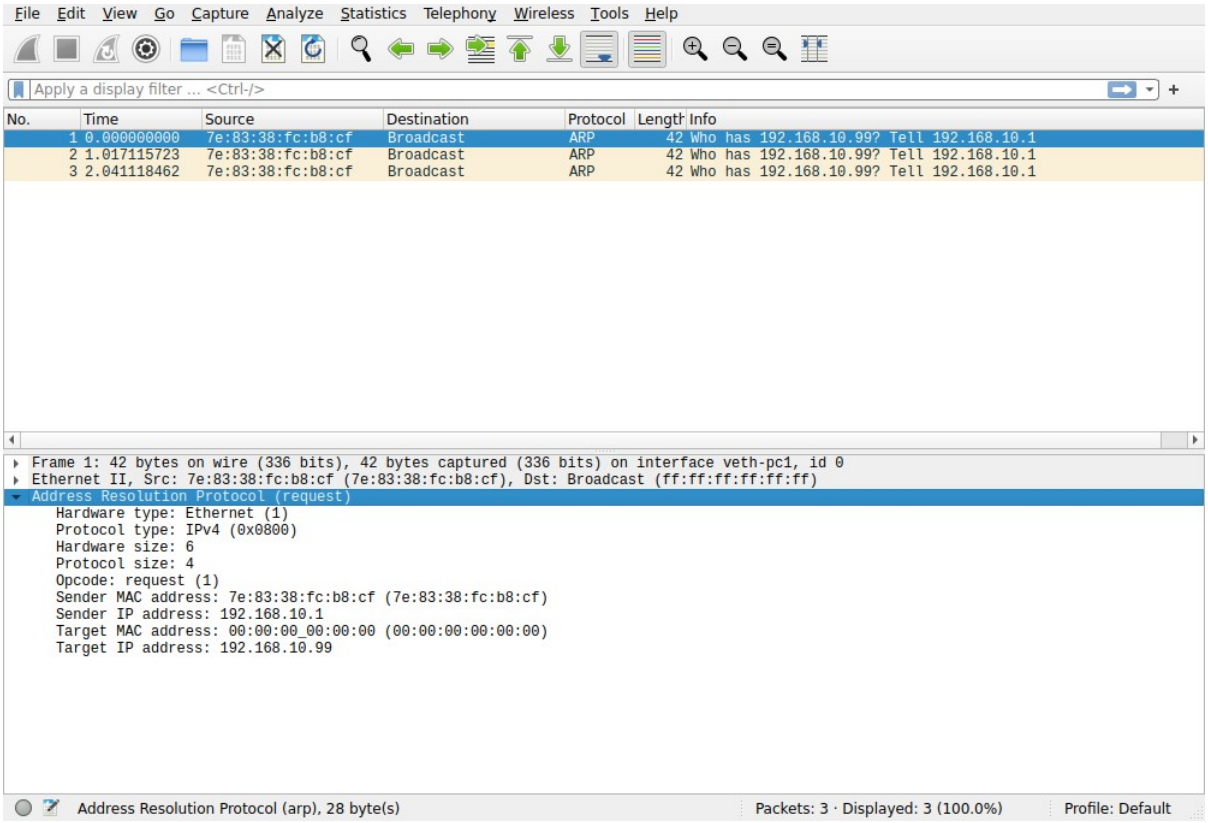


Рис. 21. *Wireshark: ARP-запити без відповіді*

Статичний запис із помилковою MAC-адресою. Після `arp -s 192.168.10.2 02:de:ad:be:ef:00` ping дає 100% втрат (рис. 22). Wireshark (рис. 23) показує, що ICMP-запити йдуть на Dst MAC `02:de:ad:be:ef:00` («no response found!»), а ARP-запитів немає: статичний запис має пріоритет, і ОС не перевіряє його правильність. Після видалення запису (кадри 4–7) динамічний ARP відновлює зв'язок. Висновок для практики: помилка в статичному записі повністю «ламає» зв'язок з вузлом і сама не виправляється.

```
maks@pc1:~$ sudo arp -s 192.168.10.2 02:de:ad:be:ef:00
maks@pc1:~$ ip neigh show
192.168.10.2 dev veth-pc1 lladdr 02:de:ad:be:ef:00 PERMANENT
maks@pc1:~$ ping -c 3 -W 1 192.168.10.2
PING 192.168.10.2 (192.168.10.2) 56(84) bytes of data.

--- 192.168.10.2 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2053ms

maks@pc1:~$ sudo arp -d 192.168.10.2
maks@pc1:~$ ping -c 1 192.168.10.2
PING 192.168.10.2 (192.168.10.2) 56(84) bytes of data.
64 bytes from 192.168.10.2: icmp_seq=1 ttl=64 time=0.091 ms

--- 192.168.10.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.091/0.091/0.091/0.000 ms
maks@pc1:~$
```

Рис. 22. *PC1: статичний запис з помилковою MAC-адресою*

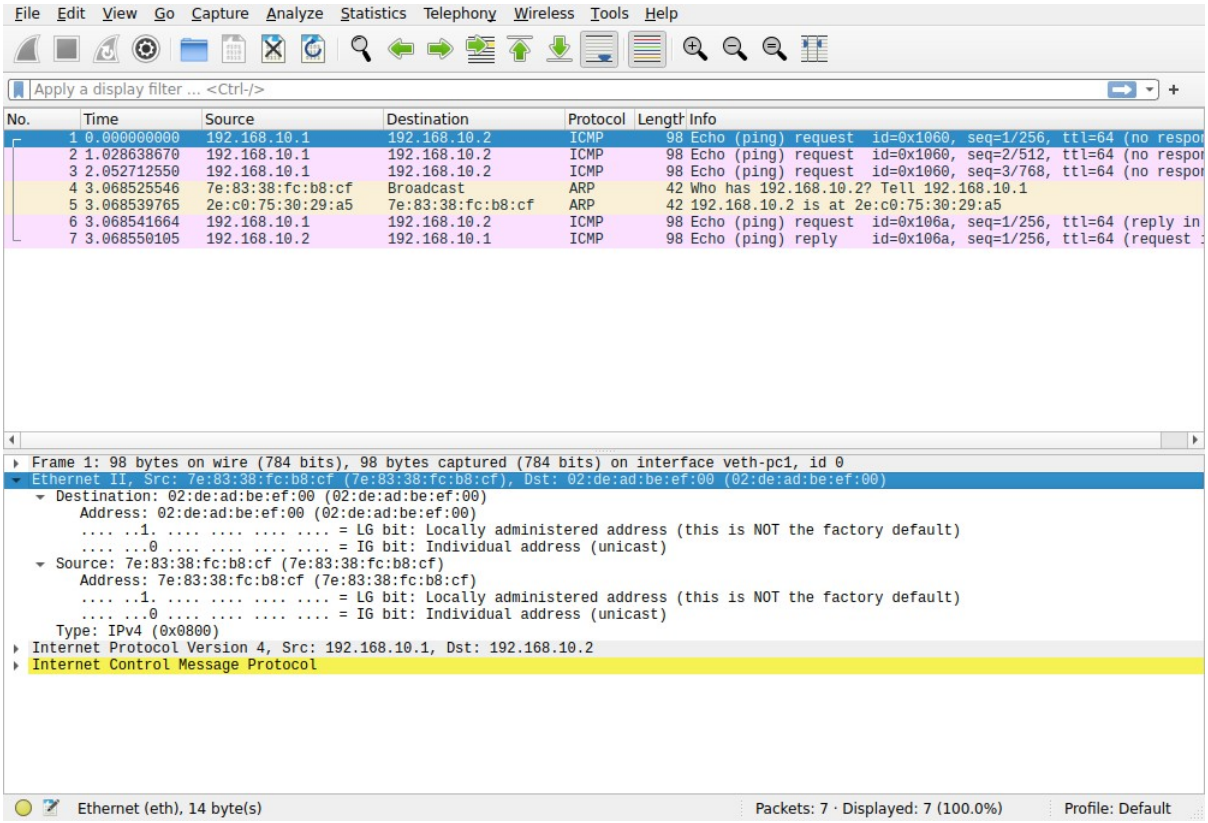


Рис. 23. *Wireshark: ICMP-кадри на неіснуючий MAC без ARP-запитів*

4. Тестова документація

4.1. Короткий тест-план

Об'єкт тестування	Реалізація протоколу ARP у мережному стеку ОС Linux (ядро 6.18): розпізнавання адрес, ARP-кеш, статичні записи
Рівень / вид	Системне функціональне тестування (чорна скринька), позитивні та негативні сценарії
Тестове оточення	Два вузли (netns pc1, pc2), з'єднані veth, підмережа 192.168.10.0/24; Wireshark 4.2.2 на PC1
Інструменти	`ping`, `arp`, `ip neigh`, `ip link`, `arping`, Wireshark/tshark
Критерій проходження	Фактичний результат кожного кроку збігається з очікуваним
Обсяг	TC-ARP-02 (варіант 2, основний), TC-ARP-00, TC-ARP-01 (варіанти 0, 1), TC-ARP-03...05 (власні)

4.2. Тест-кейси

TC-ARP-02. Статичний запис в ARP-таблиці та ping з відстеженням трафіку у Wireshark (варіант 2)

ID	TC-ARP-02
Назва	Статичний запис в ARP-таблиці та ping з відстеженням трафіку у Wireshark (варіант 2)
Пріоритет / тип	High / функціональний, позитивний
Передумови	Оточення з двох ПК у спільній підмережі 192.168.10.0/24 з'єднане (Ethernet/veth). ARP-кеші PC1 і PC2 порожні (`ip neigh show` нічого не виводить). На PC1 запущено захоплення Wireshark на інтерфейсі `veth-pc1`.
Тестові дані	IP PC2 = 192.168.10.2, MAC PC2 = 2e:c0:75:30:29:a5 IP PC1 = 192.168.10.1, MAC PC1 = 7e:83:38:fc:b8:cf

№	Дія (крок)	Очікуваний результат
1	На PC1 виконати `ip neigh show`	Вивід порожній: записів немає
2	Додати статичний запис: `sudo arp -s 192.168.10.2 2e:c0:75:30:29:a5`	Команда виконується без помилок
3	Перевірити таблицю: `arp -a` та `ip neigh show`	Є запис 192.168.10.2 → 2e:c0:75:30:29:a5 з прапорцем PERM / станом PERMANENT
4	Виконати `ping -c 4 192.168.10.2`	4 packets transmitted, 4 received, 0% packet loss
5	У Wireshark застосувати фільтр `arp || icmp`	ICMP Echo request від PC1 іде одразу, без попереднього ARP Request від PC1; Dst MAC кадру = MAC зі статичного запису
6	Застосувати фільтр `arp.opcode == 1 && arp.src.proto_ipv4 == 192.168.10.1`	Displayed: 0 — PC1 не надіслав жодного ARP-запиту
7	Повторно виконати `ip neigh show` на PC1	Запис залишився у стані PERMANENT (не старіє, не переходить у STALE)

Фактичний результат	Запис додано зі станом PERMANENT; ping — 0% втрат. У захопленні першим кадром є ICMP Echo request (Dst MAC 2e:c0:75:30:29:a5), ARP-запитів від PC1 немає (фільтр показує 0 пакетів). ARP Request/Reply (кадри 2–3) ініціює лише PC2, щоб дізнатися MAC PC1 для відповіді. Після ping запис PC1 так само PERMANENT.
Статус	PASSED
Докази	рис. 11, рис. 12, рис. 13, рис. 14

TC-ARP-00. Отримання MAC-адреси за відомою IP-адресою (варіант 0)

ID	TC-ARP-00
Назва	Отримання MAC-адреси за відомою IP-адресою (варіант 0)
Пріоритет / тип	High / функціональний, позитивний
Передумови	Оточення з двох ПК налаштоване, ARP-кеш PC1 порожній. Запущено захоплення трафіку на `veth-pc1`.
Тестові дані	Цільова IP-адреса: 192.168.10.2 (PC2)

№	Дія (крок)	Очікуваний результат
1	На PC1 виконати `arp -a`	Вивід порожній
2	Виконати `ping -c 4 192.168.10.2`	0% packet loss
3	Переглянути в Wireshark кадр ARP Request	Opcode = request (1); Dst MAC = ff:ff:ff:ff:ff:ff; Target IP = 192.168.10.2; Target MAC = 00:00:00:00:00:00
4	Переглянути кадр ARP Reply	Opcode = reply (2); unicast на MAC PC1; Sender MAC = 2e:c0:75:30:29:a5, Sender IP = 192.168.10.2
5	На PC1 виконати `arp -a`	Запис 192.168.10.2 at 2e:c0:75:30:29:a5, збігається з Sender MAC з ARP Reply

Фактичний результат	Кадр 1 — широкомовний запит «Who has 192.168.10.2? Tell 192.168.10.1», кадр 2 — відповідь «192.168.10.2 is at 2e:c0:75:30:29:a5». Після цього в кеші PC1 з'явився відповідний запис у стані REACHABLE.
Статус	PASSED
Докази	рис. 5, рис. 6, рис. 7, рис. 8

TC-ARP-01. Оновлення ARP-таблиці при отриманні ARP Reply (варіант 1)

ID	TC-ARP-01
Назва	Оновлення ARP-таблиці при отриманні ARP Reply (варіант 1)
Пріоритет / тип	High / функціональний, позитивний
Передумови	Оточення з двох ПК налаштоване, ARP-кеші порожні. Запущено захоплення трафіку на `veth-pc1`.
Тестові дані	Початковий MAC PC2 = 2e:c0:75:30:29:a5 Новий MAC PC2 = 02:aa:bb:cc:dd:02

№	Дія (крок)	Очікуваний результат
1	На PC1: `ip neigh show`, потім `ping -c 1 192.168.10.2`, знову `ip neigh show`	Після отримання ARP Reply з'являється запис 2e:c0:75:30:29:a5 REACHABLE
2	На PC2 змінити MAC: `sudo ip link set veth-pc2 address 02:aa:bb:cc:dd:02`	Інтерфейс PC2 має новий MAC
3	На PC2 надіслати ARP Reply з новим MAC: `sudo arping -U -P -c 1 -I veth-pc2 -S 192.168.10.2 192.168.10.2`	У Wireshark видно кадр ARP Reply (Opcode 2) «192.168.10.2 is at 02:aa:bb:cc:dd:02»
4	На PC1: `ip neigh show`	Запис для 192.168.10.2 оновився на 02:aa:bb:cc:dd:02 (стан STALE)
5	На PC1: `ping -c 2 192.168.10.2`	0% втрат, ICMP-кадри йдуть на новий MAC

Фактичний результат	Після ARP Reply з новим MAC запис на PC1 змінився з 2e:c0:75:30:29:a5 на 02:aa:bb:cc:dd:02 без ручного втручання, ping успішний. Wireshark позначив ситуацію як «duplicate use of 192.168.10.2 detected», бо бачив дві різні MAC для однієї IP.
Статус	PASSED
Докази	рис. 16, рис. 17, рис. 18, рис. 19

TC-ARP-03. Поведінка ARP для неіснуючого вузла (власний, негативний)

ID	TC-ARP-03
Назва	Поведінка ARP для неіснуючого вузла (власний, негативний)
Пріоритет / тип	Medium / функціональний, негативний
Передумови	Оточення налаштоване, ARP-кеш PC1 порожній, у підмережі немає вузла 192.168.10.99. Запущено захоплення трафіку.
Тестові дані	Неіснуюча IP-адреса: 192.168.10.99

№	Дія (крок)	Очікуваний результат
1	На PC1 виконати `ping -c 3 192.168.10.99`	«Destination Host Unreachable», 100% packet loss
2	Переглянути трафік у Wireshark	Лише широкомовні ARP Request «Who has 192.168.10.99?» (приблизно раз на секунду), ARP Reply немає, ICMP-кадри в мережу не виходять
3	На PC1 виконати `ip neigh show`	Запис 192.168.10.99 у стані FAILED (або INCOMPLETE під час спроб), без MAC-адреси

Фактичний результат	3 ARP Request без відповіді, ping повідомив «Destination Host Unreachable» від самого PC1, в кеші запис «192.168.10.99 dev veth-pc1 FAILED».
Статус	PASSED
Докази	рис. 20, рис. 21

TC-ARP-04. Статичний запис з помилковою MAC-адресою (власний, негативний)

ID	TC-ARP-04
Назва	Статичний запис з помилковою MAC-адресою (власний, негативний)
Пріоритет / тип	Medium / функціональний, негативний
Передумови	Оточення налаштоване, ARP-кеші порожні, запущено захоплення трафіку.
Тестові дані	IP PC2 = 192.168.10.2, помилковий MAC = 02:de:ad:be:ef:00

№	Дія (крок)	Очікуваний результат
1	`sudo arp -s 192.168.10.2 02:de:ad:be:ef:00`, потім `ip neigh show`	Запис PERMANENT з помилковим MAC
2	`ping -c 3 -W 1 192.168.10.2`	100% packet loss: кадри адресовані неіснуючому MAC, PC2 їх відкидає
3	Переглянути трафік у Wireshark	ICMP Echo request з Dst MAC 02:de:ad:be:ef:00, жодного ARP Request (статичний запис має пріоритет), відповідей немає
4	Видалити запис `sudo arp -d 192.168.10.2` і повторити `ping -c 1`	Динамічне ARP-розпізнавання відновлює зв'язок: 0% втрат

Фактичний результат	Три ICMP-запити пішли на 02:de:ad:be:ef:00 без жодної спроби ARP і лишилися без відповіді. Після видалення запису ОС виконала ARP Request/Reply і ping пройшов.
Статус	PASSED
Докази	рис. 22, рис. 23

TC-ARP-05. Видалення та очищення записів ARP-кешу (власний)

ID	TC-ARP-05
Назва	Видалення та очищення записів ARP-кешу (власний)
Пріоритет / тип	Medium / функціональний
Передумови	В ARP-кеші PC1 є динамічний запис для 192.168.10.2.
Тестові дані	IP PC2 = 192.168.10.2

№	Дія (крок)	Очікуваний результат
1	`sudo arp -d 192.168.10.2`, потім `arp -a`	Запис видалено
2	`ping -c 1 192.168.10.2`, потім `ip neigh show`	Запис створено знову (REACHABLE)
3	`sudo ip neigh del 192.168.10.2 dev veth-pc1`, потім `ip neigh show`	Запис видалено
4	Створити запис ping-ом, виконати `sudo ip neigh flush all`, потім `ip neigh show`	Усі динамічні записи видалено
5	Додати статичний запис, виконати `sudo ip neigh flush all`, потім `ip neigh show`	Статичний (PERMANENT) запис НЕ видаляється командою flush
6	`sudo arp -d 192.168.10.2`, потім `ip neigh show`	Статичний запис видалено

Фактичний результат	Усі кроки дали очікуваний результат. Підтверджено важливу особливість: `ip neigh flush all` не видаляє статичні записи, їх треба видаляти явно (`arp -d` або `ip neigh del`).
Статус	PASSED
Докази	рис. 10, рис. 15

4.3. Зведення результатів тестування

ID	Назва	Тип	Статус
TC-ARP-02	Статичний запис в ARP-таблиці та ping з відстеженням трафіку у Wireshark (варіант 2)	функціональний, позитивний	PASSED
TC-ARP-00	Отримання MAC-адреси за відомою IP-адресою (варіант 0)	функціональний, позитивний	PASSED
TC-ARP-01	Оновлення ARP-таблиці при отриманні ARP Reply (варіант 1)	функціональний, позитивний	PASSED
TC-ARP-03	Поведінка ARP для неіснуючого вузла (власний, негативний)	функціональний, негативний	PASSED
TC-ARP-04	Статичний запис з помилковою MAC-адресою (власний, негативний)	функціональний, негативний	PASSED
TC-ARP-05	Видалення та очищення записів ARP-кешу (власний)	функціональний	PASSED

Виконано 6 тест-кейсів: 6 пройдено (PASSED), 0 провалено. Дефектів у реалізації ARP не виявлено.

5. Висновки

У ході лабораторної роботи досліджено протокол ARP, що працює на межі канального та мережного рівнів моделі OSI. Для цього створено ізольоване тестове оточення з двох вузлів, з'єднаних віртуальним Ethernet-каналом, в якому весь трафік належить до експерименту.

Експериментально підтверджено механізм отримання MAC-адреси за IP-адресою: вузол надсилає широкомовний ARP Request (Dst `ff:ff:ff:ff:ff:ff`, Target MAC нульовий), власник адреси відповідає одноадресним ARP Reply, і лише після цього починається передавання IP-пакетів. Розібрано поля ARP-пакета (28 байтів, у кадрі 42 байти) і показано, що одержувач запиту заздалегідь кешує адресу відправника, а застарілі записи перевіряються одноадресними запитами (стани STALE → DELAY → PROBE).

Опановано засоби ОС для роботи з ARP-кешем: перегляд (`arp -a`, `ip neigh show`), видалення (`arp -d`, `ip neigh del`), очищення (`ip neigh flush all`) і додавання статичних записів (`arp -s`). Встановлено, що flush не видаляє статичні записи. За основним завданням (варіант 2) показано, що зі статичним записом вузол не надсилає ARP-запитів і одразу використовує задану MAC-адресу. Негативний тест показав зворотний бік: помилковий статичний запис повністю блокує зв'язок і не виправляється автоматично.

Навчилися аналізувати трафік у Wireshark: застосовувати фільтри (`arp`, `arp.opcode`, `arp.src.proto_ipv4`), розбирати заголовки Ethernet та ARP, використовувати експертні попередження (виявлення дублювання IP-адреси, що є ознакою можливого ARP spoofing).

Розроблено тестову документацію: короткий тест-план і 6 тест-кейсів (основний для варіанта 2, кейси для варіантів 0 і 1 та 3 власні, зокрема 2 негативні) з передумовами, тестовими даними, кроками, очікуваними й фактичними результатами та посиланнями на докази. Усі тест-кейси пройдено успішно.